
FastAPI

Python Framework

FastAPI

- Any machine learning model's end goal is a deployment for production purposes.
- Building a [REST API](#)(Application Programming Interface) is the best possible way to evaluate model performance.
- In python, Django and more evidently [Flask](#) frameworks are used for this purpose.
- For machine learning, Flask is preferred more than Django.
- Here comes FastAPI which is faster than Flask, providing higher performance boost, easier to code, comes with automatic documentation, provides data validation on input data provided to the server, and many more such features .

FastAPI..

- FastAPI can handle 9000 requests at a time. FastAPI can manage database sessions, web sockets, easy GraphQL injection and many more are still being built.
- FastAPI is being used by tech giants such as Netflix, Facebook, Microsoft, Uber.
- Making production-ready RestAPIs with few lines of code is apprehensive.

Software Dependencies that are required for FastAPI:-

- FastAPI is built upon two major python libraries – Starlette(for web handling) and Pydantic(for data handling).
- Python 3.6+
- FastAPI
- ASGI server for production such as Uvicorn or Hypercorn.
- Pydantic

How to install FastAPI?

We can install FastAPI using pip

- `pip install fastapi`
- `pip install uvicorn`

Basic Hello World

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def index():
    return {"message": "Hello World"}
```

Save this program in main.py then from a terminal, execute:
uvicorn main:app --reload

Basic Hello World

The following message should be printed:

```
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)  
INFO: Started reloader process [28720]  
INFO: Started server process [28722]  
INFO: Waiting for application startup.  
INFO: Application startup complete.
```

After redirecting to the webpage, it should appear as:

```
{ "message" : "Hello World" }
```

Interactive Documentation

<http://127.0.0.1.8000/docs> will show the interactive documentation by Swagger UI.

YAML

- YAML is a data serialization format designed for human readability and interaction with scripting languages.
- PyYAML is a YAML parser and emitter for Python.
- PyYAML features a complete YAML 1.1 parser, Unicode support, pickle support, capable extension API, and sensible error messages.
- PyYAML supports standard YAML tags and provides Python-specific tags that allow to represent an arbitrary Python object.

Swagger UI

- Swagger UI allows anyone — be it your development team or your end consumers — to visualize and interact with the API's resources without having any of the implementation logic in place.
- It's automatically generated from your OpenAPI (formerly known as Swagger) Specification, with the visual documentation making it easy for back end implementation and client side consumption.

Image classification API

- Data annotation (with Unsplash API + Labelme)
- Model Training (With Tensorflow)
- Making the API (With Uvicorn and FastApi)
- Deploying the API on a remote server (With Docker and Google Cloud Platform)
- In this project we will try to classify an input image into four classes:
 - City
 - Beach
 - Sunset
 - Trees/Forest

MODEL

- We build a classifier using Tensorflow. We will use MobileNet_V2 as the backbone of the classifier since it is fast and less likely to over-fit given the tiny amount of labeled samples we have, you can easily use it by importing it from keras_applications

```
from tensorflow.keras.applications import MobileNetV2
```

```
base_model = MobileNetV2(include_top=False, input_shape=input_shape,  
weights=weights)
```

Transfer learning

- Commonly used idea is to tackle the lack of labeled samples is to use transfer learning.
- It is when you transfer some of the weights learned from a source task (like image classification with a different set of labels) to your target task as the starting point of your training.
- This allows for a better initialization compared to starting from random and allows for reusing some of the representations learned on the source task for our multi-label classification.

```
base_model = MobileNetV2(include_top=False, input_shape=input_shape,  
weights="imagenet")
```

Training

- We split the dataset into two folds, training and validation and use the binary_crossentropy as our target along with the binary_accuracy as the evaluation metric.

configuration files:

data_config.yaml for defining the classes and input size

input_shape: [null, null, 3]

resize_shape: [224, 224]

images_base_path: '../example/data/'

targets: ['beach', 'city', 'sunset', 'trees']

image_name_col: 'name'

training_config.yaml for defining some training parameters

use_augmentation: true

batch_size: 32

epochs: 1000

initial_learning_rate: 0.0001

model_path: "image_classification.h5"

Making the API

- We will be using FastAPI to expose a predictor through an easy to use API that can take as input an image file and outputs a JSON with the classification scores for each class.
- Predictor class that can easily load a tensorflow.keras model and have a method to classify an image that is in the form of a file object.

Model.py

```
# config.yaml
resize_shape: [224, 224]
targets: ['beach', 'city', 'sunset', 'trees']
model_path: "image_classification.h5"
```

The main file of our API becomes trivial when using FastAPI:

```
from fastapi import FastAPI, File, UploadFile
from fast_image_classification.predictor import ImagePredictor
app = FastAPI()
predictor_config_path = "config.yaml"
predictor = ImagePredictor.init_from_config_url(predictor_config_path)
@app.post("/scorefile/")
def create_upload_file(file: UploadFile = File(...)):
    return predictor.predict_from_file(file.file)
```

Thanks!!

Questions?